



High-Performance E-Commerce Backend Engine

مشروع البرمجة المتوازية

Django + PostgreSQL + Redis + Celery + Nginx	التقنية المستخدمة
https://github.com/ibrahimhana/high-performance-e-commerce-engine/tree/ibrahim-branch	رابط المشروع
ابراهيم هنا - ناجة الحلبي - سلمى جعفر - محمد خالد عبد المجيد	أسماء الطلاب

مقدمة

يهدف المشروع إلى بناء محرك Backend عالي الأداء لنظام تجارة إلكترونية، مع التركيز على تحمل الطلبات المتزامنة، حماية البيانات المشتركة، تحسين زمن الاستجابة باستخدام Redis، ضمان سلامة المعاملات، اختبار النظام تحت الضغط، وتحليل الاختناقات قبل وبعد التحسين وهذه الطلبات الخمس الأخيرة.

Requirement 6 — Distributed Caching

تقليل الضغط على قاعدة البيانات وتحسين زمن الاستجابة للطلبات المتكررة.

استخدمنا Redis لتخزين بيانات المنتجات المتكررة. عند طلب تفاصيل منتج، يتم البحث أولاً في Redis. إذا كانت البيانات موجودة يتم إرجاعها مباشرة. إذا لم تكن موجودة يتم جلبها من قاعدة البيانات وتخزينها في Redis.

`apps/catalog/cache_services.py`

`docker-compose.yml`

```
najat@MacBook-Air high-performance-final-copy % docker compose ps
```

NAME	STATUS	IMAGE	COMMAND	SERVICE	CREATED
hpe_celery_beat	Up About a minute	hpe_app:latest	"/app/entrypoint.sh ..."	celery_beat	About a minute ago
hpe_celery_worker	Up About a minute	hpe_app:latest	"/app/entrypoint.sh ..."	celery_worker	About a minute ago
hpe_nginx	Up About a minute	nginx:1.27-alpine	"/docker-entrypoint.sh ..."	nginx	About a minute ago
hpe_postgres	Up About a minute (healthy)	postgres:16-alpine	"docker-entrypoint.s..."	postgres	About a minute ago
hpe_redis	Up About a minute (healthy)	redis:7-alpine	"docker-entrypoint.s..."	redis	About a minute ago
hpe_web1	Up About a minute	hpe_app:latest	"/app/entrypoint.sh ..."	web1	About a minute ago
hpe_web2	Up About a minute	hpe_app:latest	"/app/entrypoint.sh ..."	web2	About a minute ago
hpe_web3	Up About a minute	hpe_app:latest	"/app/entrypoint.sh ..."	web3	About a minute ago

```
najat@MacBook-Air high-performance-final-copy %
```

```
najat@MacBook-Air high-performance-final-copy % docker compose logs web1 web2 web3 | grep CACHE
```

hpe_web1	[2026-06-19 20:29:59,247]	INFO	apps.catalog.cache	inst=web1 ::	CACHE MISS	->	product:1
hpe_web2	[2026-06-19 20:30:14,972]	INFO	apps.catalog.cache	inst=web2 ::	CACHE MISS	->	product:1
hpe_web3	[2026-06-19 20:30:33,727]	INFO	apps.catalog.cache	inst=web3 ::	CACHE MISS	->	product:1
hpe_web3	[2026-06-19 20:42:25,827]	INFO	apps.catalog.cache	inst=web3 ::	CACHE HIT	->	product:1
hpe_web1	[2026-06-19 20:30:41,167]	INFO	apps.catalog.cache	inst=web1 ::	CACHE MISS	->	product:1
hpe_web2	[2026-06-19 20:42:22,244]	INFO	apps.catalog.cache	inst=web2 ::	CACHE HIT	->	product:1
hpe_web1	[2026-06-19 20:41:21,209]	INFO	apps.catalog.cache	inst=web1 ::	CACHE MISS	->	product:1

توضح الصورة أن أول طلب على المنتج كان Cache Miss، أي أنه تم جلب البيانات من قاعدة البيانات. بعد ذلك أصبحت الطلبات Cache Hit، أي أن البيانات أصبحت تُقرأ من Redis مباشرة، وهذا يثبت تطبيق المتطلب السادس الخاص بالتخزين المؤقت.

Requirement 7 — Concurrency Control

منع تضارب البيانات عند تحديث المخزون من عدة طلبات متزامنة.

تم استخدام Pessimistic Locking باستخدام `select_for_update`، و Optimistic Locking باستخدام `conditional update` و `version`.

`apps/orders/services.py`

`scripts/optimistic_lock_demo.py`

```
najat@MacBook-Air high-performance-final-copy %  
docker compose exec -T -e BASE_URL=http://nginx web1 python scripts/optimistic_lock_demo.py  
-> logging in as 'demo' on http://nginx  
-> product RACE-001 id=3 starting_stock=50  
-> firing 100 optimistic checkouts on 50 threads  
mode OPTIMISTIC (CAS + retry)  
requests_fired 100  
successful_sales (HTTP 201) 50  
out_of_stock (HTTP 409) 50  
optimistic_conflicts (HTTP 409) 0  
other_errors 0  
stock_before 50  
stock_after 0  
version_after 51  
consistent True  
saw_real_contention True  
Expected: 50 sales, stock 0, version_after >= 50.
```

توضح الصورة نتيجة اختبار التزامن باستخدام Optimistic Locking. تم إرسال طلبات شراء متزامنة على نفس المنتج، ونجح فقط العدد الموافق للمخزون المتاح، بينما تم رفض الطلبات الزائدة. ظهور `consistent = True` يثبت أن النظام حافظ على صحة المخزون ومنع Race Condition.

Requirement 8 — ACID/ TRANSACTION INTEGRITY

ضمان أن الدفع، تحديث المخزون، وإنشاء الطلب تتجج كلها أو تفشل كلها.

تم وضع عملية checkout داخل `transaction.atomic`. إذا فشلت عملية الدفع أو أي خطوة أخرى، يتم rollback ولا يتم حفظ تغييرات ناقصة.

`apps/orders/services.py`

`apps/orders/payments.py`

`scripts/acid_demo.py`

```
najat@MacBook-Air high-performance-final-copy % docker compose exec -T -e BASE_URL=http://nginx web1
python scripts/acid_demo.py
-> logging in as 'demo' on http://nginx
-> product RACE-001 id=3
-> baseline {'stock': 50, 'orders_for_demo_user': 50}

=== Phase A: 30 declined ===
HTTP: {402: 30} (expected {402: 30})
delta: {'stock': 0, 'orders_for_demo_user': 0} (expected 0/0)
PASS

=== Phase B: 30 gateway errors ===
HTTP: {502: 30} (expected {502: 30})
delta: {'stock': 0, 'orders_for_demo_user': 0} (expected 0/0)
PASS

=== Phase C: 30 approved ===
HTTP: {201: 30} (expected {201: 30})
delta: {'stock': -30, 'orders_for_demo_user': 30} (expected stock -30, orders +30)
PASS

ALL PHASES PASSED - atomic rollback holds.
najat@MacBook-Air high-performance-final-copy %
```

توضح الصورة نجاح اختبار سلامة المعاملات. تم اختبار حالات فشل الدفع، خطأ بوابة الدفع، وحالة الدفع الناجح. عند الفشل لم يتم حفظ طلب ناقص ولم يتم تخفيض المخزون بشكل خاطئ، وعند النجاح تم تنفيذ العملية كاملة. هذا يثبت تحقيق مبدأ Atomicity ضمن ACID.

Requirement 9 -Stress Testing

إثبات أن النظام يتحمل 100 مستخدم متزامن دون انهيار أو فقدان بيانات.

تم استخدام Locust لمحاكاة traffic حقيقي يشمل تصفح المنتجات وعمليات checkout.

loadtest/locustfile.py

docs/STRESS_TEST_REPORT.md

loadtest/report.html

```
[2026-06-19 21:11:21,239] c6df76478b39/INFO/locust.main: --run-time limit reached, shutting down
[2026-06-19 21:11:21,307] c6df76478b39/INFO/locust.main: writing html report to file: /mnt/locust/report.html
[2026-06-19 21:11:21,309] c6df76478b39/INFO/locust.main: Shutting down (exit code 0)
Type  Name  # reqs  # fails | Avg  Min  Max  Med | req/s  failures/s
-----|-----|-----|-----|-----|-----|-----|-----|-----|-----
GET    GET catalog list  1096  0(0.00%) | 4    4    1    20 | 4 | 18.31  0.00
GET    GET my orders     210  0(0.00%) | 7    7    2    30 | 7 | 3.51   0.00
GET    GET product detail 2245  0(0.00%) | 3    3    0    133 | 3 | 37.50  0.00
GET    GET top-products  556  0(0.00%) | 2    2    1    11  | 3 | 9.29   0.00
POST   POST checkout     323  0(0.00%) | 121  2    740 | 100 | 5.40   0.00
POST   setup: register   100  0(0.00%) | 179  125  458 | 150 | 1.67   0.00
-----|-----|-----|-----|-----|-----|-----|-----|-----|-----
Aggregated  4530  0(0.00%) | 16    0    740 | 4 | 75.67  0.00
```

توضح الصورة نتيجة اختبار الضغط باستخدام Locust لمحاكاة عدد كبير من المستخدمين المتزامنين. تم قياس عدد الطلبات، الفشل، معدل الطلبات في الثانية، وزمن الاستجابة، وذلك لإثبات أن النظام يستطيع العمل تحت الضغط دون انهيار.

Requirement 10 - Benchmarking

قياس الأداء، تحديد Bottleneck، ثم إثبات التحسين بالأرقام.

تم استخدام benchmark script لقياس زمن الاستجابة. تم تحديد أن cache hit كان ما زال يسبب queries على قاعدة البيانات. تم تحسين استراتيجية الكاش باستخدام مفتاح مباشر `{product:{id}}`، وتأجيل views_count إلى Redis.

`scripts/benchmark.py`

`docs/BENCHMARK_REPORT.md`

`apps/catalog/cache_services.py`

`apps/catalog/tasks.py`

```
scripts/benchmark.py --url http://nginx/api/catalog/products/1/ --requests 300 --concurrency 1 --label AFTER
label='AFTER' url=http://nginx/api/catalog/products/1/ requests=300 concurrency=1

## Benchmark - AFTER
- URL: `http://nginx/api/catalog/products/1/`
- Total requests: 300 (concurrency 1)
- Wall time: 0.23 s
- Throughput: **1279.1 req/s**
- Status codes: {200: 300}

| metric | latency (ms) |
|-----|-----:|
| avg    | 0.77         |
| p50    | 0.76         |
| p90    | 0.83         |
| p95    | 0.86         |
| p99    | 0.90         |
| max    | 1.26         |
```

توضح الصورة نتيجة قياس الأداء بعد التحسين. تم إرسال 300 طلب إلى endpoint تفاصيل المنتج، وتم قياس زمن الاستجابة باستخدام مؤشرات avg و p50 و p95 و p99. تساعد هذه النتائج على إثبات تحسن الأداء بعد تطبيق Redis Cache وتحسين استراتيجية الوصول إلى البيانات.

5. Results

Raw numbers

metric	BEFORE	AFTER	Δ
avg latency	9.09 ms	5.82 ms	-36 %
p50	8.04 ms	4.10 ms	-49 %
p90	9.82 ms	6.58 ms	-33 %
p95	10.75 ms	7.66 ms	-29 %
p99	17.03 ms	9.71 ms	-43 %
max	360 ms	342 ms	-5 % (noise)
throughput	543 req/s	848 req/s	+56 %
HTTP failures	0	0	-